

Internet of Things - CAI26442

Final Project Report

Smart Agriculture Monitoring Using Virtual Sensors, MQTT, Gateway Processing, and Real-Time Alerts

Prepared by: Group X

Members: [Student 1], [Student 2], [Student 3 if applicable]

Submission Date: May 21, 2026

Project Scope: A fully virtual IoT system that simulates temperature and humidity sensors, publishes readings in JSON format through MQTT, processes them in a Python gateway, and displays real-time monitoring results with alert conditions.

Member	Main Responsibility	Editable Placeholder
Student 1	Publisher and sensor simulation	Replace with real name/ID
Student 2	Gateway, JSON validation, alert logic	Replace with real name/ID
Student 3	Subscriber, testing, report/presentation	Delete if not applicable

Table of Contents

- 1. Introduction
- 2. Project Objectives
- 3. System Architecture
- 4. System Components
- 5. Data Format and JSON Design
- 6. MQTT Topic Design
- 7. Gateway Processing and Decision Logic
- 8. Results and Screenshots
- 9. Challenges and Lessons Learned
- 10. Conclusion

1. Introduction

The purpose of this project is to design and implement a virtual Internet of Things (IoT) system without using physical hardware. The selected application is Smart Agriculture Monitoring. The system simulates environmental sensing in an agricultural zone using virtual temperature and humidity sensors. Data is produced by a Python publisher, transmitted through a public MQTT broker, processed by a Python gateway, and presented in real time by a subscriber application.

This design demonstrates the essential components of an IoT pipeline: sensing, message formatting, publish/subscribe communication, gateway-level processing, monitoring, and rule-based alerts. The implementation follows the required architecture and includes editable source code, report, presentation, and visual evidence artifacts.

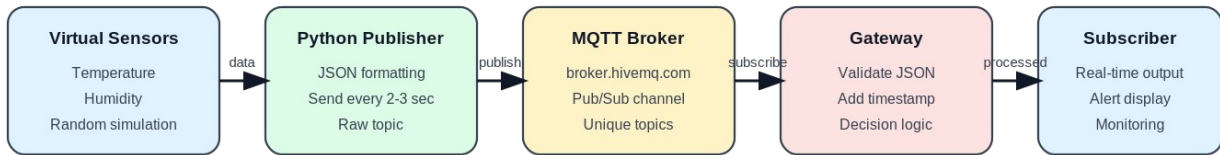
2. Project Objectives

- Simulate at least two different virtual sensors: temperature and humidity.
- Represent each reading as valid JSON including device_id, sensor values, and status.
- Publish sensor data over MQTT every 2-3 seconds using a unique project topic.
- Build a gateway that validates JSON messages, adds a UTC timestamp, and applies decision rules.
- Create a subscriber that shows real-time processed messages and alert conditions.
- Document the design, implementation, results, and lessons learned in a professional report.

3. System Architecture

The project follows the required pipeline: Virtual Sensors -> Python Publisher -> MQTT Broker -> Gateway -> Subscriber / Real-Time Monitoring. Two MQTT topics are used: one for raw sensor data and one for processed gateway output.

Smart Agriculture Monitoring - Required IoT Pipeline



Raw topic: iot/project/groupX/agriculture/raw | Processed topic: iot/project/groupX/agriculture/processed

Figure 1. System architecture of the Smart Agriculture Monitoring project.

4. System Components

Component	File / Tool	Role	Key Output
Virtual Sensors	publisher.py	Generates random temperature and humidity values.	Raw sensor readings
Publisher	publisher.py	Formats data as JSON and publishes every 2-3 seconds.	Raw MQTT messages
MQTT Broker	broker.hivemq.com	Routes messages between publishers and subscribers.	Pub/Sub transport
Gateway	gateway.py	Validates JSON, adds timestamp, applies decision logic.	Processed JSON messages
Subscriber	subscriber.py	Displays real-time data and alert status.	Readable terminal monitoring

5. Data Format and JSON Design

The publisher sends valid JSON to satisfy the data modeling requirement. The raw message includes device_id, temperature, humidity, and status. The gateway enriches this structure by adding gateway_timestamp_utc, gateway_id, and alert_reason, then publishes the processed record.

```
{
  "device_id": "groupX_agri_node_01",
  "temperature": 33.2,
  "humidity": 63,
  "status": "RAW",
  "publisher_time_utc": "2026-05-16T10:15:00+00:00"
}
```

Processed gateway output example:

```
{
  "device_id": "groupX_agri_node_01",
  "temperature": 33.2,
  "humidity": 63.0,
  "status": "ALERT",
  "alert_reason": "High temperature",
  "gateway_timestamp_utc": "2026-05-16T10:15:02+00:00",
  "gateway_id": "groupX_gateway_01"
}
```

6. MQTT Topic Design

The design uses a public MQTT broker and two group-specific topics. The GroupX placeholder should be replaced with the actual group identifier before final submission to preserve uniqueness.

Topic	Direction	Purpose
iot/project/groupX/agriculture/raw	Publisher -> Gateway	Raw simulated sensor data

iot/project/groupX/agriculture/
processed

Gateway -> Subscriber

Validated, timestamped, alert-
classified data

7. Gateway Processing and Decision Logic

The gateway is responsible for processing data before it reaches the monitoring layer. It rejects invalid JSON payloads, checks mandatory fields and value ranges, adds a UTC timestamp, and calculates the final monitoring status. This guarantees that only validated, decision-ready information is forwarded to the subscriber.

Rule	Condition	Status	Reason
Rule 1	temperature >= 32 C	ALERT	High temperature
Rule 2	humidity >= 80%	ALERT	High humidity
Rule 3	temperature >= 29 C or humidity <= 35%	WARNING	Moderate risk requiring attention
Default	No rule triggered	NORMAL	Safe environmental range

```

if temperature >= 32 or humidity >= 80:
    status = "ALERT"
elif temperature >= 29 or humidity <= 35:
    status = "WARNING"
else:
    status = "NORMAL"

```

8. Results and Screenshots

The included screenshots are editable project visuals prepared for the report package. Because live MQTT execution depends on an external broker connection, they should be replaced with screenshots captured from the final local run before submission if the instructor requires direct evidence of execution.

```

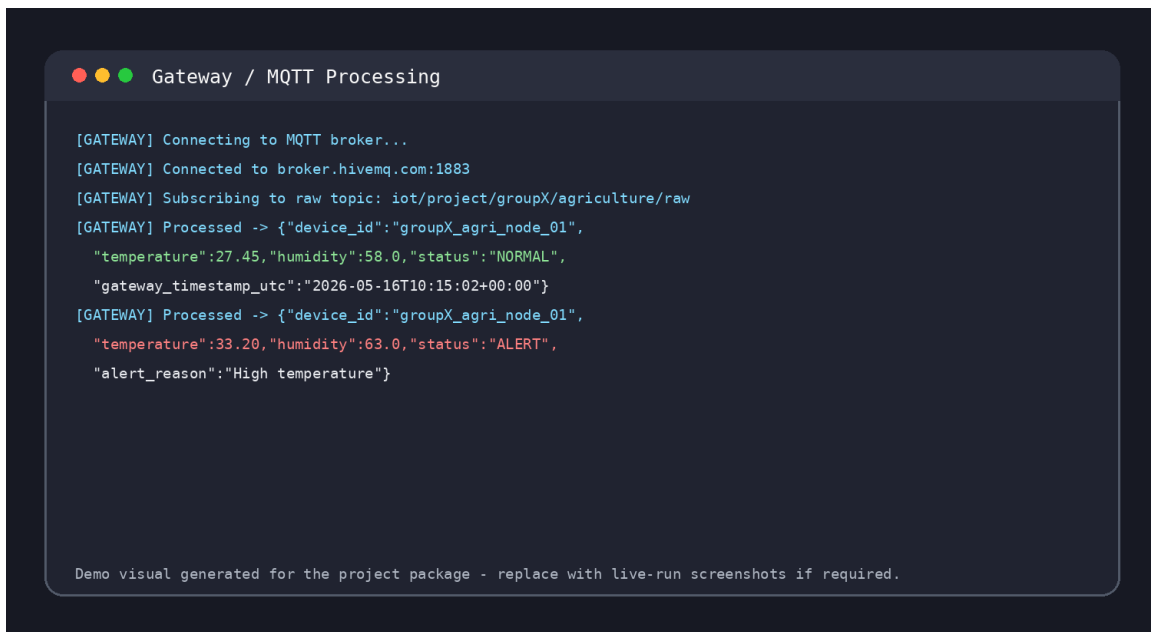
Publisher Running

[PUBLISHER] Connecting to MQTT broker...
[PUBLISHER] Connected to broker.hivemq.com:1883
[PUBLISHER] Topic: iot/project/groupX/agriculture/raw
[PUBLISHER] Device ID: groupX_agri_node_01
[PUBLISHER] Sent -> {"device_id":"groupX_agri_node_01",
  "temperature":27.45,"humidity":58,"status":"RAW"}
[PUBLISHER] Sent -> {"device_id":"groupX_agri_node_01",
  "temperature":33.20,"humidity":63,"status":"RAW"}

Demo visual generated for the project package - replace with live-run screenshots if required.

```

Figure 2. Publisher terminal showing repeated JSON publication.



```
Gateway / MQTT Processing

[GATEWAY] Connecting to MQTT broker...
[GATEWAY] Connected to broker.hivemq.com:1883
[GATEWAY] Subscribing to raw topic: iot/project/groupX/agriculture/raw
[GATEWAY] Processed -> {"device_id":"groupX_agri_node_01",
  "temperature":27.45,"humidity":58.0,"status":"NORMAL",
  "gateway_timestamp_utc":"2026-05-16T10:15:02+00:00"}
[GATEWAY] Processed -> {"device_id":"groupX_agri_node_01",
  "temperature":33.20,"humidity":63.0,"status":"ALERT",
  "alert_reason":"High temperature"}
```

Demo visual generated for the project package - replace with live-run screenshots if required.

Figure 3. Gateway processing, validation, timestamps, and status assignment.

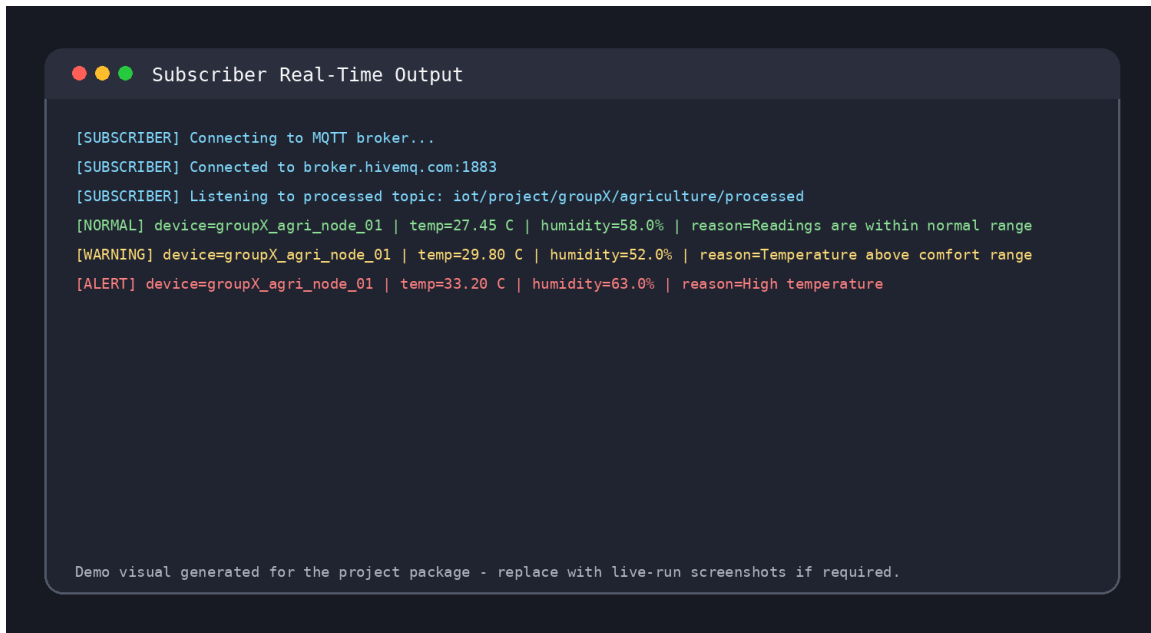


Figure 4. Subscriber displaying real-time processed messages.

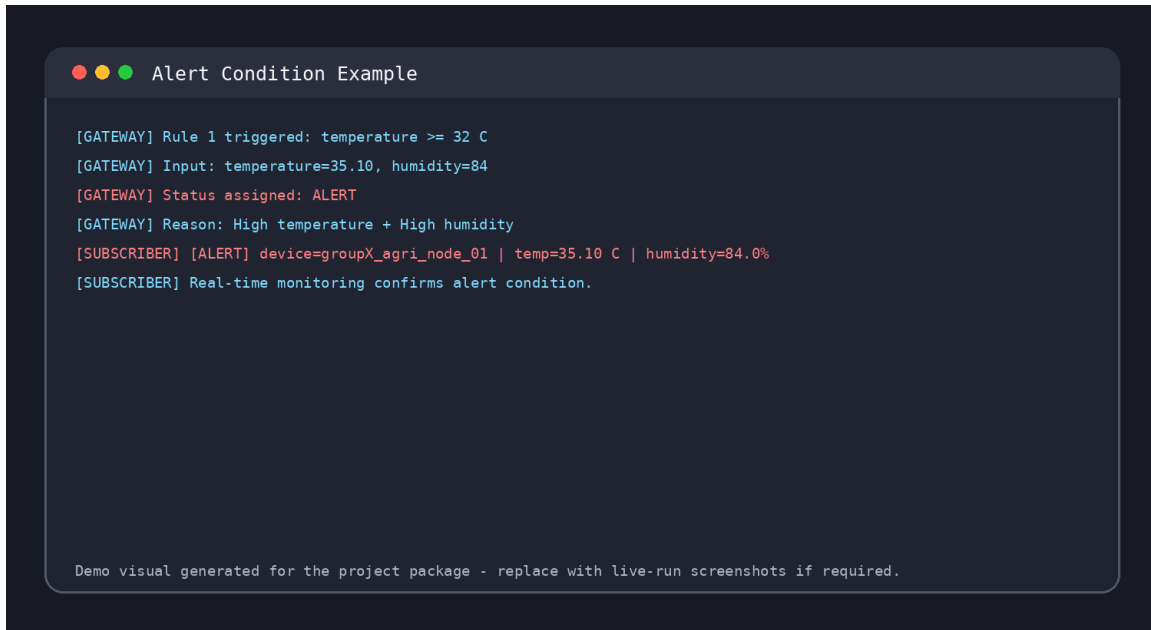


Figure 5. Alert condition produced by threshold-based decision rules.

9. Challenges and Lessons Learned

Challenge	Risk	Mitigation
Unique MQTT topics	Public brokers may receive many unrelated messages.	Use a group-specific topic path.
JSON validation	Malformed or incomplete payloads can break processing.	Reject messages missing required fields.
Real-time timing	Messages should be frequent but not excessive.	Publish every 2-3 seconds as required.

Alert design

Rules must be simple and observable.

Use thresholds that reliably trigger NORMAL, WARNING, and ALERT.

10. Conclusion

This project satisfies the required final-project elements for a virtual IoT monitoring system. It implements two sensors, JSON data modeling, MQTT publishing and subscription, a Python gateway, real-time monitoring output, a unique topic structure, and multi-rule alert logic. The package also contains editable source files, an editable report, an editable presentation, and evidence images to support the final submission workflow.

Appendix A. Execution Commands

```
pip install -r requirements.txt
python gateway.py
python subscriber.py
python publisher.py
```

Run the commands in separate terminal windows from the Source_Code folder. Replace GroupX in the topic names with the actual group identifier before the final demonstration.